World Action Models for Cross Embodiment Robot Control

D0 High Level System Design | Joshua Jerin and Krrish Thakku Suresh
Advisor: Huu Tri Nguyen | CS 5001 | October 2, 2026

1 Title, goal statement, and conventions

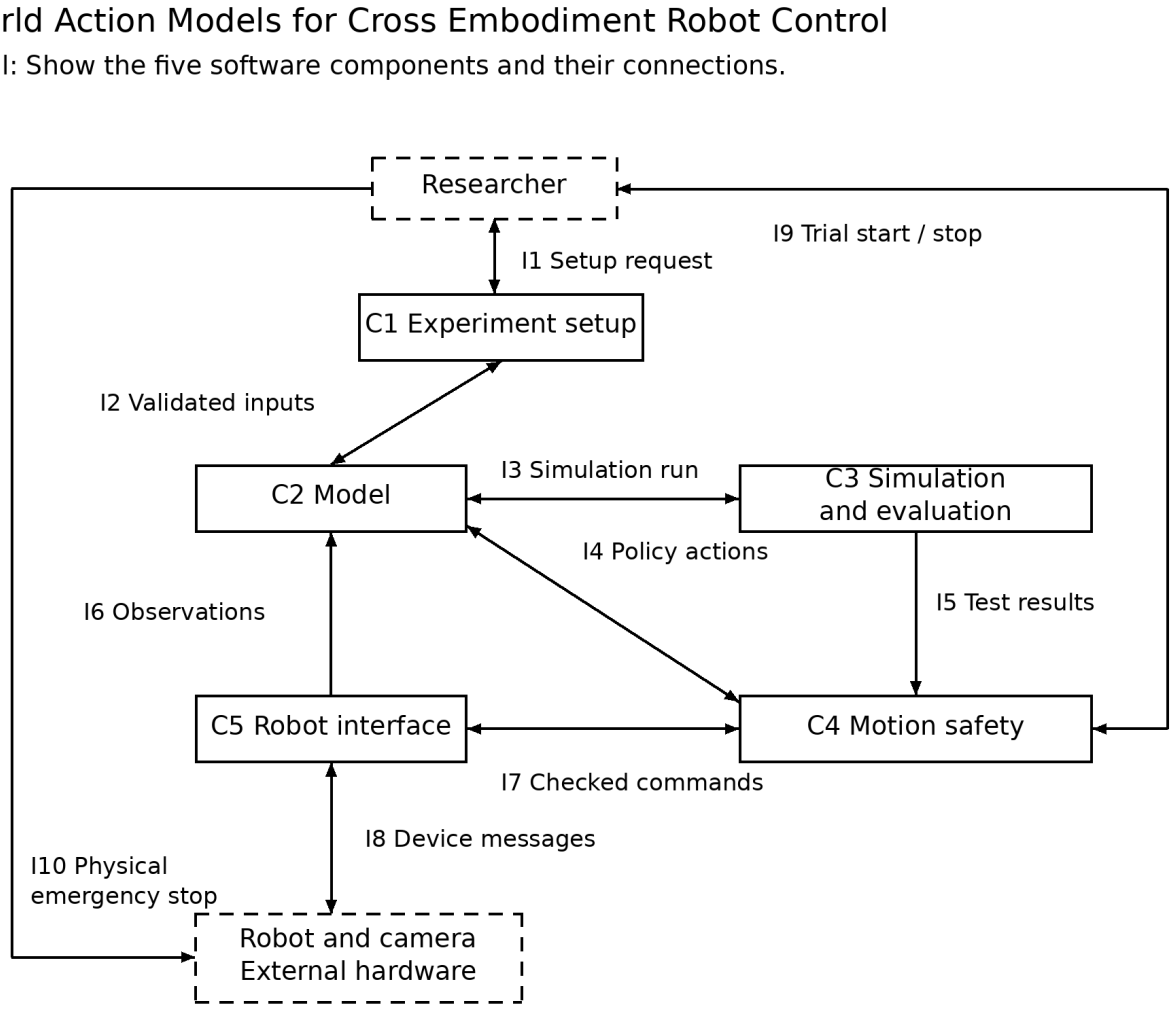
Our goal is to adapt a pretrained world action model to a new robot arm with a small set of demonstrations, compare its performance across robots, and test it safely in a controlled lab setting.

Inputs are a pretrained model, demonstrations, a robot description, and live camera/robot-state data. Outputs are an adapted model, simulation results, checked robot commands, and a trial log.

This is a proposed design. The robot, model, simulator, and numerical limits still need team and advisor approval. C means component. I means interface: a connection between two parts. For example, I9 is the trial start/stop request. Names beside the IDs match the connection table in Section 4.

2 Block diagram (the D0 diagram)

Figure 1. System structure



Legend: Solid boxes = project software; dashed boxes = researcher or hardware.
Arrows = data or control; two arrowheads = both directions.
The physical emergency stop bypasses the software.

3 Component responsibility table

The owner column is a suggested division for the team to confirm. Each component saves its own outputs as ordinary project files; a separate storage service is unnecessary for this first version.

Component	Responsibility	Connections in / out	Proposed owner
C1 Experiment setup	Validate the researcher's dataset and robot setup before starting an experiment.	In/out: I1, I2	Joshua
C2 Model	Adapt the pretrained model or use a saved model to predict actions.	In: I2, I3, I4, I6 Out: I2, I3, I4	Krrish
C3 Simulation and evaluation	Test the model in simulation and compare its success rate with a single-embodiment baseline.	In/out: I3 Out: I5	Krrish
C4 Motion safety	Check that a physical run is allowed and keep each command within the approved limits.	In: I4, I5, I7, I9 Out: I4, I7, I9	Joshua
C5 Robot interface	Translate observations and checked commands between our software and the robot.	In: I7, I8 Out: I6, I7, I8	Krrish

Connection to the submitted user stories

US-01, adapt with few demonstrations: C1 checks the data and embodiment; C2 fine-tunes the model and saves its dataset/configuration references.

US-02, compare seen and unseen robots with a single-embodiment baseline: C3 runs matched tests using C2 and returns the report through C1. The report separates robots used in training from held-out robots.

US-03, protect people nearby: C4 checks workspace and speed limits; C5 sends only checked commands. The lab must keep people outside the approved motion area and provide a working emergency stop.

US-04, protect hardware: C4 rejects or limits unsafe actions before C5 sends them. C3 supplies the simulation evidence needed before a physical run.

US-05, protect camera privacy: C1 excludes or anonymizes identifiable images before normal dataset storage; C5 keeps live images temporary unless they go through that review.

This also covers every component: C1 supports US-01/02/05; C2 supports US-01/02; C3 supports US-01/02/04; C4 supports US-03/04; C5 supports US-03/04/05.

A few operating rules

Before training, C1 rejects demonstrations with the wrong action dimension; C2 asks C3 to load and check the robot description through I3. C2 saves the adapted checkpoint with the dataset, robot description, settings and code version. C3 saves the test report. C4 saves the predicted action, sent action and any limit event for each physical step.

C4 runs separately from the model process. C5 is the only software component allowed to send robot commands. C5 forwards C4's heartbeat unchanged to the controller, whose independent timeout must stop the robot if it disappears. Verify this on the actual hardware before a physical trial.

4 Interface specification table

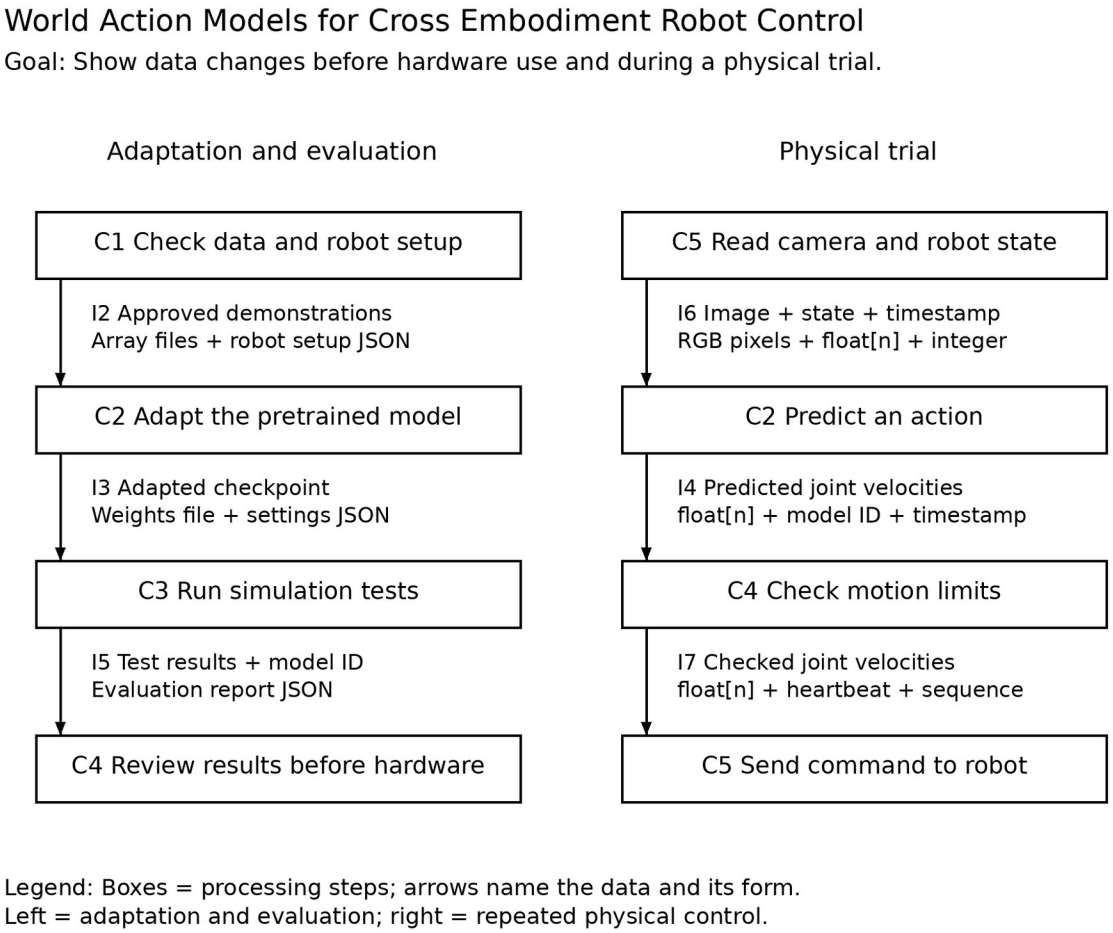
I1–I10 match Figure 1. JSON is used for small configuration/status records; images, arrays and model weights remain files or memory buffers. Here, n is the registered joint count. float[n] means n finite numbers in the agreed joint order. Joint positions use radians and joint velocities use radians per second. IDs/paths are strings; counts and monotonic timestamps (ms) are integers; success rates are 0–1 floats. Status is an enum: ready, running, complete, failed or stopped. RGB pixels are uint8[H,W,3]; image size, joint count and motion limits come from robot/model setup JSON. A heartbeat is an increasing integer. Local process messages use JSON over Unix sockets.

Connection	**Inputs and outputs; format and method**	**If something fails**
I1 Setup request Researcher ↔ C1	Dataset/model paths and robot setup JSON → validation/status. JSON configuration through a command-line tool.	C1 lists invalid files/settings and does not start the run.
I2 Validated inputs C1 ↔ C2	Approved demonstration arrays + robot/model settings → saved model/test-report paths and run status. Local function/job call with JSON and file references.	C2 stops on load/memory errors; partial checkpoints are not treated as completed output.
I3 Simulation run C2 ↔ C3	Robot description + checkpoint/settings → validation and test result. During tests: RGB image + joint-state float[n] ↔ velocity float[n]. Local simulator/model calls.	C3 reports failed or incomplete episodes; incomplete tests cannot authorize hardware.
I4 Policy actions C2 ↔ C4	C4 sends run permission/stop; C2 sends model ID, velocity float[n], sequence and observation time. JSON messages over a local socket.	C4 rejects the wrong model, stale data, wrong dimensions or nonfinite values and stops motion.
I5 Test results C3 → C4	Model/robot IDs, episode counts, observed success rates and test status. Versioned JSON report file.	C4 refuses a physical run if the report is missing, incomplete, mismatched or below the approved threshold.
I6 Observations C5 → C2	Timestamped RGB image and measured joint positions/velocities float[n]. Shared memory plus local metadata message.	C5 reports sensor failure; C2 rejects invalid/stale input. No fresh action means the physical run stops.
I7 Checked commands C4 ↔ C5	Limited velocity float[n] + sequence + safety heartbeat → joint positions/velocities float[n], timestamp and acknowledgement bool. JSON messages over a local socket.	C5 refuses expired/replayed commands; C4 stops on stale feedback or failed acknowledgement.
I8 Device messages C5 ↔ robot/camera	Checked velocities + unchanged safety heartbeat → acknowledgement bool, joint-state float[n] and RGB pixels. Wired device API; exact USB/Ethernet protocol follows hardware selection.	C5 stops on lost communication. The controller must time out if C4's heartbeat stops; C5 must not generate it.
I9 Trial start/stop Researcher ↔ C4	Selected model, robot description, approved limits and report-reviewed bool and E-stop-tested bool → allowed/refused/stopped status. Command-line request with JSON settings.	C4 refuses missing checks. Stop requests disable motion; a fault requires a new session check.
I10 Physical emergency stop Researcher → robot	Pressed E-stop switch → latched robot stop. Manufacturer's physical stop circuit, independent of our program.	No physical trial until this stop is tested. Reset follows the lab's procedure.

I7 example for a two-joint test fixture: {"sequence":12,"joint_velocity":[0.02,-0.01],"sent_ms":1000,"heartbeat":61,"valid_for_ms":50}. The vector must match the arm. The timeout is illustrative and needs hardware validation.

5 Data-flow diagram

Figure 2. Offline and physical data flows



C3 exchanges simulated observations and actions with C2 over I3. In a physical trial, C5 returns robot state to C4 over I7. C4 checks the planned motion and stopping distance; if it cannot verify a safe correction, it stops.

Timing and test targets

Target from the previous submission	**How this design handles it**
50 demonstrations; 50 evaluation episodes; 60% success threshold	C1 checks data; C3 compares the new arm with a single-embodiment baseline; C4 checks the report before hardware. These remain draft targets.
0.25 m/s end-effector speed; at least 100 physical control steps	C4 checks motion limits and logs predicted and sent actions. Incomplete logs mean incomplete tests.
Stop within 100 ms if safety heartbeats cease	Use the controller's independent timeout and measure actual stopping time. Hardware stays disabled until the lab confirms this target is achievable.

Proposed 50 ms control-cycle budget: sensing 10 ms, prediction 30 ms, motion checks 5 ms, delivery 5 ms. Reject actions older than 50 ms; send a safety heartbeat every 20 ms. These are untested budgets, subject to model/controller measurements and lab approval. Keep training off the live-inference GPU.

6 Architecture pattern and justification

We use the Week 5 pipeline pattern for adaptation and evaluation, and the embedded sensor-actuator pattern for physical control. The data-preparation steps run in sequence; the robot repeatedly senses, predicts, checks and acts.

Required criterion	**Reason for the choice**
Fit to the problem	The pipeline follows the path from demonstrations to a tested model. The sensor-actuator loop matches repeated robot control.
Team skills	Local modules let us divide model work, evaluation and integration. Joshua's submitted background in backend work, automation and debugging fits setup and safety checks; the proposed ownership still needs team agreement.
Performance and timing	Training stays outside the physical loop. Local communication avoids internet delay, and the selected controller must support the independent stop timeout.
Scalability	The first version uses one active robot and small experiments. We can add datasets and robot descriptions without introducing network services.
Hardware constraints	Use existing team/UC hardware and a pretrained model that fits its memory. Keep robot-specific commands inside C5.

We rejected microservices because separate network services, deployment and distributed failures add work for a two-person project without helping the initial one-robot experiment.

Week 3 constraints: adapt an existing open-source model rather than fund training from scratch; use available team/UC compute; test in simulation first; keep only necessary data and avoid identifiable imagery. Private local/UC storage is our proposed hosting choice. Keyboard commands, readable text output and labels that do not depend on color address accessibility. No exact budget, hosting provider or accessibility standard was set in the earlier submission.

7 Decision log

Choice	**Alternative**	**Reason**
Adapt a pretrained model.	Train a foundation model from scratch.	Fits the project's compute limits and the few-demonstration goal.
Use five local components.	Deploy each part as a network service.	Keeps ownership and interfaces clear without unnecessary deployment work.
Check commands outside the model process.	Send model predictions directly to the robot.	Preserves workspace/speed checks when the model behaves incorrectly.
Keep reviewed data and versioned result files.	Retain all video and overwrite checkpoints.	Supports privacy and lets us reproduce which data and settings produced a result.

References: submitted project background, constraints and user stories; Week 5 architecture lecture; Assignment 5 instructions. Story links follow the five stated user stories, with simulation approval supporting safety stories US-03/04.